# UNIVERSITY OF RUHUNA

## BACHELOR OF INFORMATION AND COMMUNICATION TECHNOLOGY DEGREE

## LEVEL II (Semester I)

### ICT2132- Object Oriented Programming Practicum

### Mini Project Report

**Submitted By: Group 20- OOPify**

**TG/2023/1759**

**TG/2023/1753**

**TG/2023/1709**

# Table of Contents


1. Introduction.....3
2. Objectives.....4
3. System Overview.....4
4. Implementation.....5
   4.1 Object-Oriented Programming Concepts.....5
   4.2 Database Handling.....6
   4.3 GUI Development.....6
   4.4 Authentication System.....6
   4.5 Attendance Management.....6
   4.6 Marks and GPA Calculation.....7
5. Technologies Used.....7
6. User Interfaces.....7
7. Problems and Solutions.....10
   7.1 Debugging Errors.....10
   7.2 Database Issues.....11
   7.3 Complexity of System Design.....11
   7.4 Team Coordination.....11
8. Conclusion.....11
9. References.....11
10. Contribution.....12

# 1. Introduction

This mini project was developed as part of the ICT2132 – Object-Oriented Programming Practicum offered by the Department of Information and Communication Technology, Faculty of Technology, University of Ruhuna. The main objective of this project is to design and implement a Faculty Management System that supports and automates essential academic and administrative activities within the faculty.

The system is designed to handle various operations such as managing user profiles, course information, undergraduate attendance, medical records, marks, notices, and timetables in a structured and efficient way. By converting these manual processes into a digital system, it helps reduce paperwork, minimize errors, and improve the overall efficiency of data management.

The application was developed using Java as the main programming language, while JavaFX was used to design a modern and user-friendly graphical interface. IntelliJ IDEA was used as the development environment to ensure better project organization and easier debugging. MySQL was selected as the database management system, and JDBC was used to connect the application with the database for efficient data storage and retrieval.

During the development of this system, key Object-Oriented Programming (OOP) concepts such as encapsulation, inheritance, polymorphism, and abstraction were applied to build a well-structured and maintainable application. In addition, proper error and exception handling techniques were implemented to ensure system stability and smooth operation. This project provided valuable practical experience in developing a real-world software application while improving our understanding of OOP concepts, database integration, GUI design, and teamwork.

## 2. Objectives

• To design and develop a complete faculty management system using Java
• To implement role-based access control
• To manage attendance, marks, and GPA
• To automate eligibility checking
• To ensure secure data storage using MySQL
• To apply OOP concepts
• To develop a user-friendly interface
• To gain teamwork experience

## 3. System Overview

The Faculty Management System is designed to handle multiple types of users with different functionalities.

**Admin**

The Admin is responsible for managing the overall system. The main functionalities include:

- Creating and managing user accounts
- Managing courses and assigning lecturers
- Creating and updating notices
- Managing timetables
- Performing CRUD operations for all system data

**Lecturer**

Lecturers interact mainly with academic data. Their responsibilities include:

- Updating personal profile (except login credentials)
- Uploading and updating course materials
- Entering and managing student marks
- Viewing student attendance, medical records, and eligibility
- Viewing notices

**Technical Officer**

Technical Officers mainly handle attendance and medical data:

- Recording attendance for students
- Managing medical records
- Updating profile information
- Viewing timetables and notices

**Undergraduate**

Undergraduates are the primary users who access their own academic data:

- Viewing attendance details
- Viewing marks, grades, and GPA
- Viewing timetables and notices
- Updating personal details

# 4. Implementation

The system was implemented using Java as the programming language and MySQL as the database. The development followed a modular approach where each feature was implemented as a separate component.

### 4.1 Object-Oriented Programming Concepts

The system strongly demonstrates OOP principles:

- **Encapsulation**: Data is protected within classes using private variables and accessed through methods
- **Inheritance**: Common features are shared among different user classes
- **Polymorphism**: Methods behave differently based on the object context
- **Abstraction**: Complex processes are hidden, and only essential features are exposed

These concepts improved code reusability and maintainability.

### 4.2 Database Handling

- MySQL is used to store all system data
- A separate database connection class is used
- Prepared Statements are used to prevent SQL injection
- Data includes users, courses, attendance, marks, medical records, and notices

### 4.3 GUI Development

The system provides a graphical user interface for better usability. Each user role has a separate dashboard with relevant functionalities.

Key features:

- Easy navigation
- Form-based input system
- Clear layout for viewing data
- Error messages for invalid inputs

### 4.4 Authentication System

A secure login system is implemented:

- Users log in using credentials
- System identifies the user role
- Redirects to the appropriate dashboard
- Prevents unauthorized access

### 4.5 Attendance Management

- Attendance is recorded for both theory and practical sessions
- Each subject includes multiple sessions
- Medical records can be added for absences
- System calculates attendance percentage
- Different cases are handled (above 80%, below 80%, with/without medical)

### 4.6 Marks and GPA Calculation

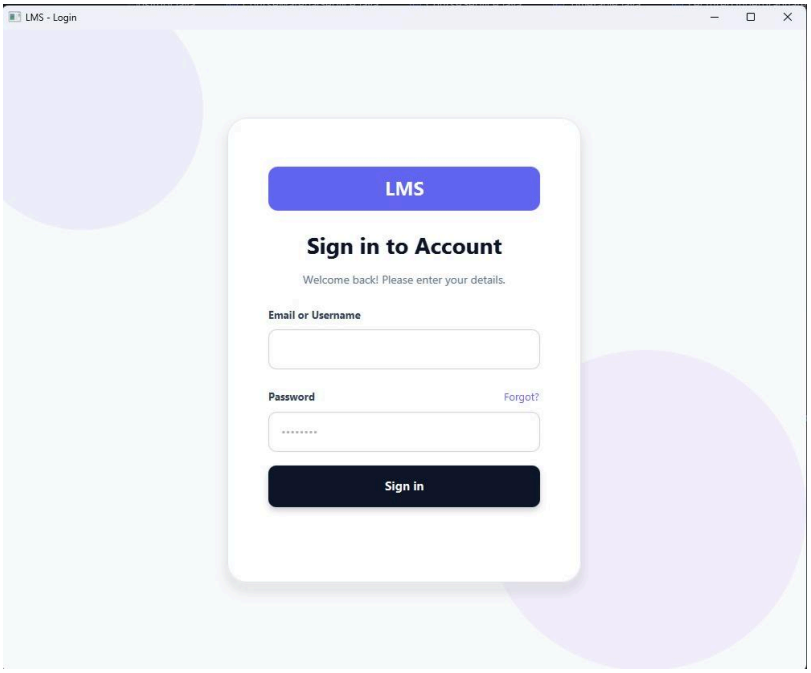
- Marks are recorded out of 100
- Continuous Assessment (CA) marks are evaluated
- Eligibility is checked (CA ≥ 40%)
- Final marks are calculated
- Grades are assigned based on standard criteria
- SGPA and CGPA are automatically calculated

## 5. Technologies Used

- ➢ Programming Language: Java
- ➢ Database: MySQL
- ➢ GUI Tools: JavaFX
- ➢ IDE: IntelliJ IDEA
- ➢ Version Control: GitHub

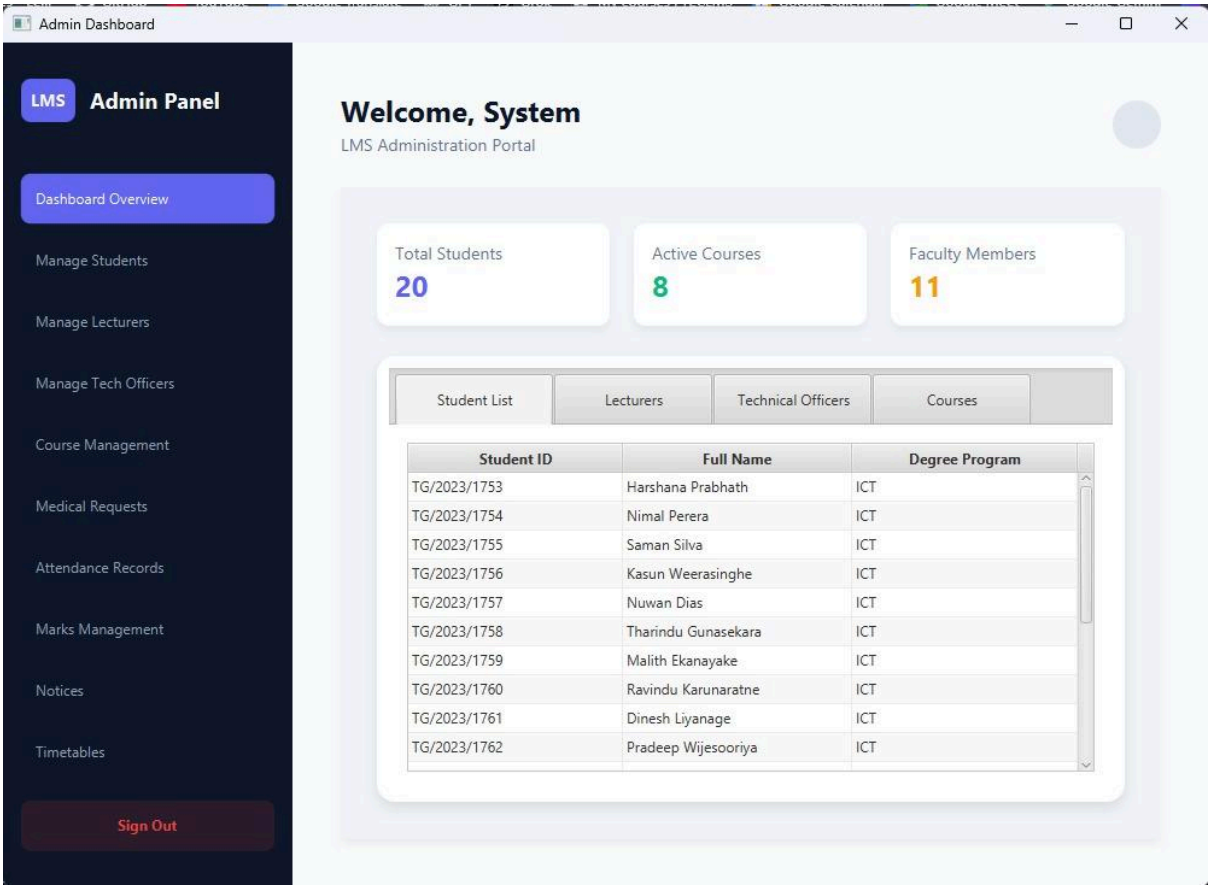
## 6. User Interfaces

### Login Interface



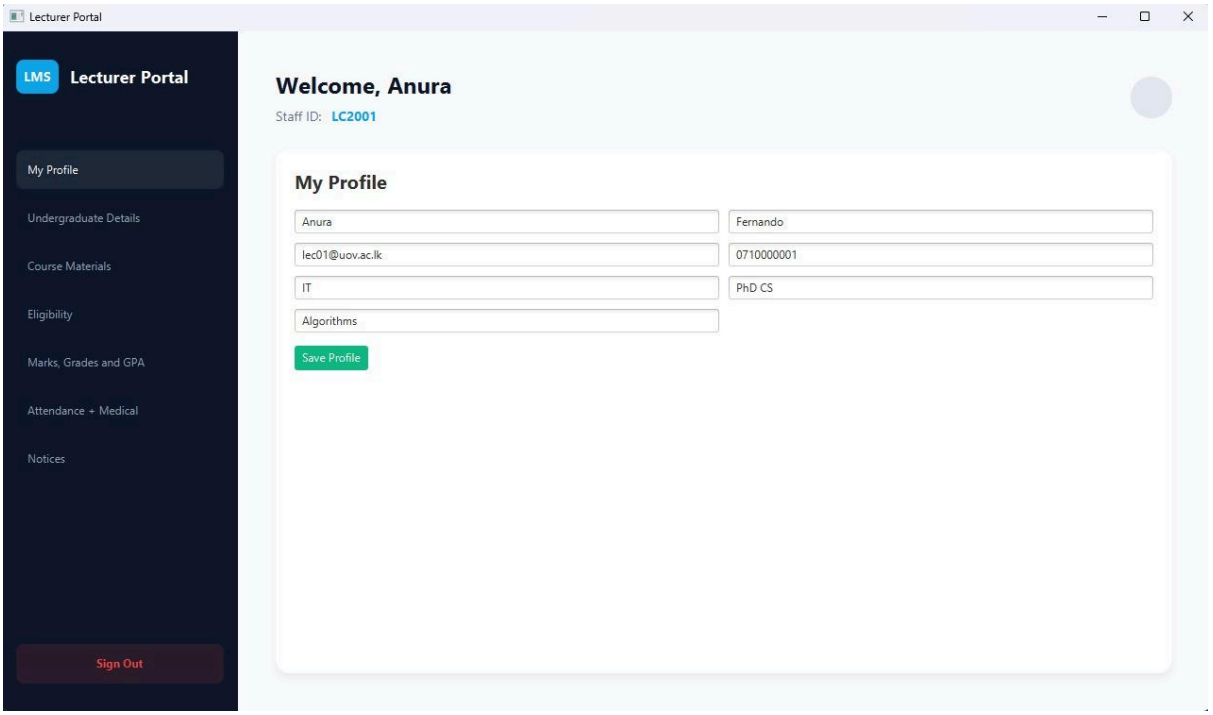
**Admin Interface**

- Dashboard overview
- User management panel
- Course and timetable management
- Notice management



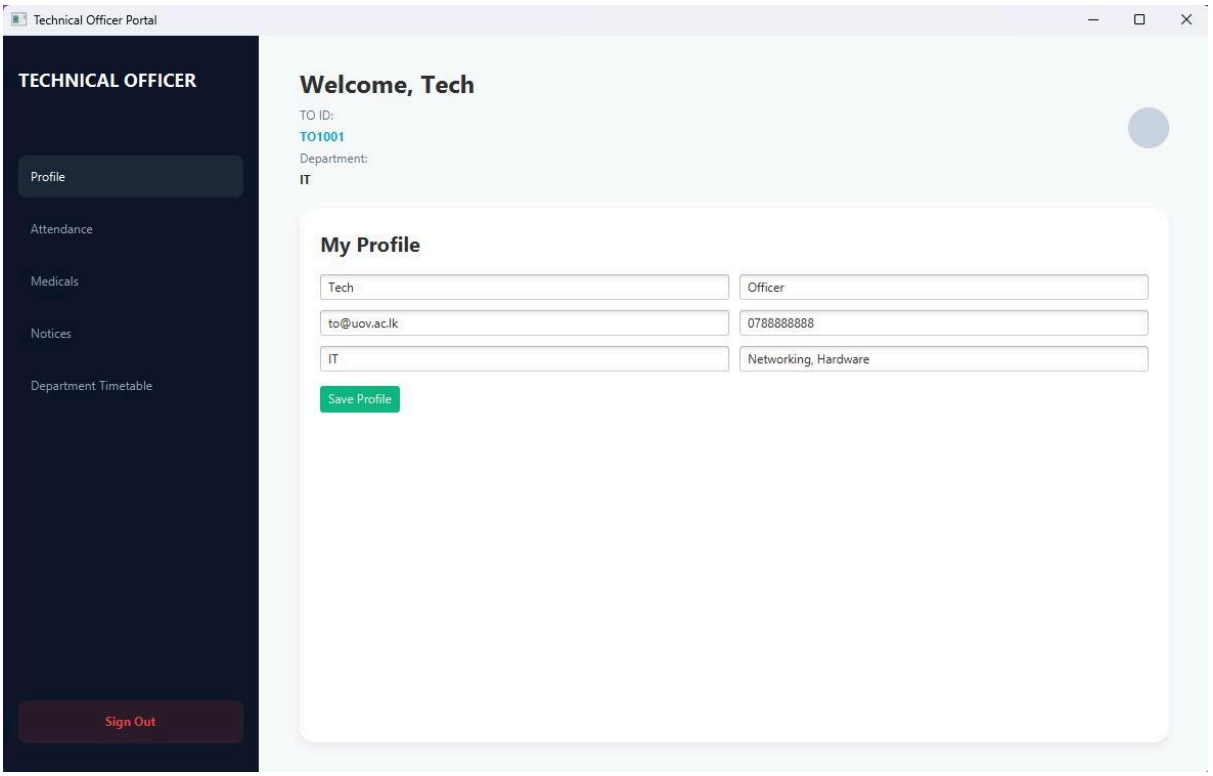
## Lecturer Interface

- Profile management
- Marks entry system
- Student details view



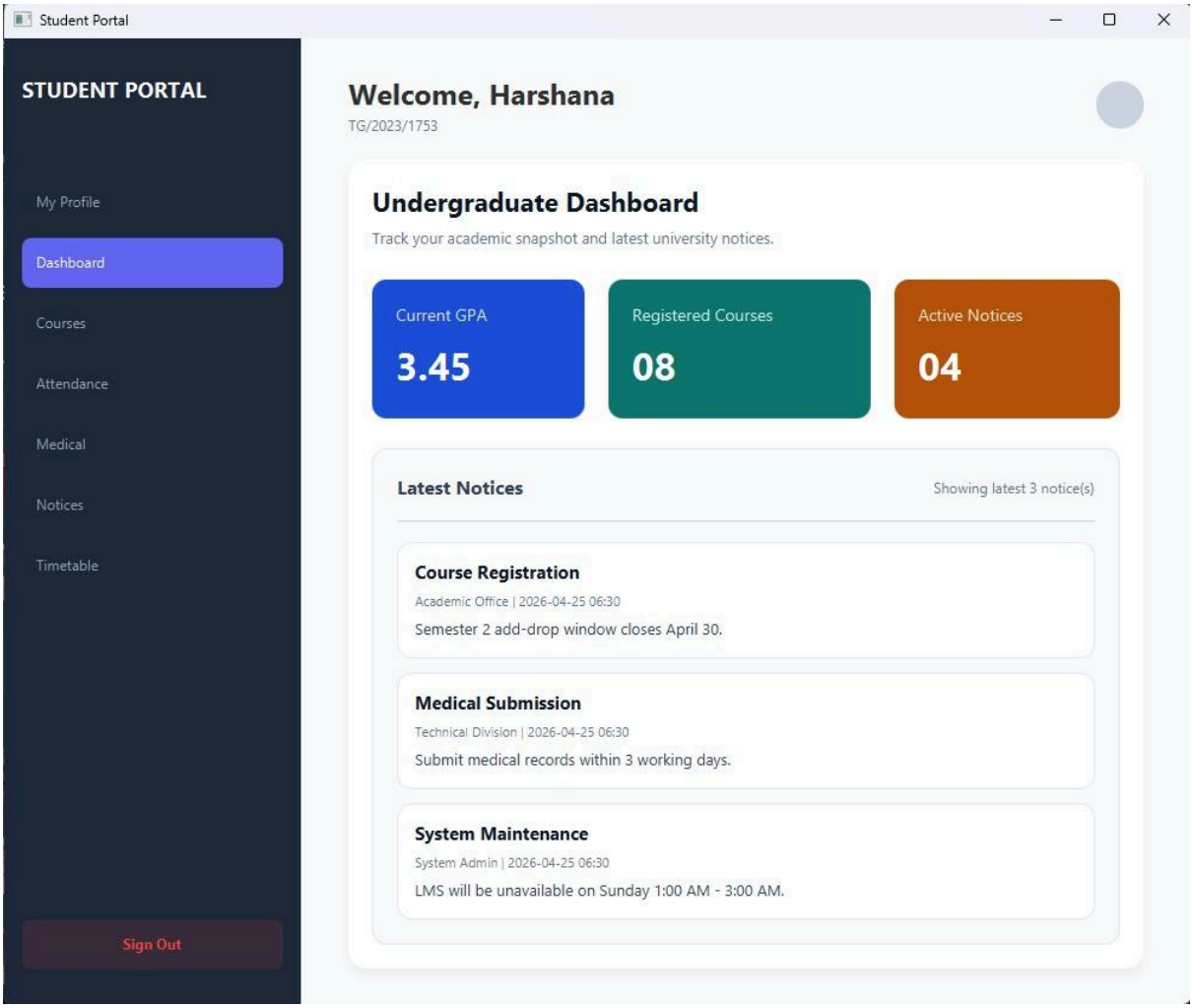
## Technical Officer Interface

- Attendance recording system
- Medical record management



**Undergraduate Interface**

- Academic details view
- GPA and grades display
- Timetable and notices



# 7. Problems and Solutions

### 7.1 Debugging Errors

- ➔ Finding logical and runtime errors was difficult.
  Solution: Used debugging tools, proper testing, and meaningful error messages.

### 7.2 Database Issues

- ➔ Connection errors and incorrect queries occurred.
  Solution: Improved query handling, used prepared statements, and tested database connections.

### 7.3 Complexity of System Design

- ➔ Designing multiple modules was challenging.
  Solution: Divided the system into smaller components and implemented step by step.

### 7.4 Team Coordination

- ➔ Managing tasks among members was sometimes difficult.
  Solution: Clear task division and regular communication improved teamwork.

# 8. Conclusion

The Faculty of Technology Management System successfully fulfills the requirements of the ICT2132 mini project. It provides an efficient solution for managing academic and administrative activities.

The system demonstrates strong use of OOP concepts and integrates database management with a user-friendly interface. It improves accuracy, reduces manual workload, and ensures better organization of academic data.

Through this project, valuable experience was gained in software development, problem-solving, and teamwork.

# 9.References


1. Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.
2. Chacon, S., & Straub, B. (2019). Pro Git (2nd ed.). Apress.
3. Ebbers, D., & Vos, J. (2020). Learn JavaFX 17. Apress.
4. Fowler, M. (2018). UML Distilled (3rd ed.). Addison-Wesley Professional.
5. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (2017). Design Patterns. Addison-Wesley.
6. GeeksforGeeks. (2025). Object-Oriented Programming (OOPs) Concept in Java. Retrieved from https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/
7. JetBrains. (2025). IntelliJ IDEA Documentation. Retrieved from https://www.jetbrains.com/idea/
8. Oracle Corporation. (2023). Java Documentation. Retrieved from https://docs.oracle.com/javase/tutorial/
9. Oracle Corporation. (2023). JavaFX Documentation. Retrieved from https://openjfx.io/
10. Oracle Corporation. (2023). MySQL Documentation. Retrieved from https://dev.mysql.com/doc/
11. Reese, G. (2019). JDBC Pocket Guide (2nd ed.). O'Reilly Media. Schildt, H. (2022). Java: The Complete Reference (12th ed.). McGraw-Hill Education.
12. Tahaghoghi, S. M. M., & Williams, H. E. (2019). Learning MySQL (2nd ed.). O'Reilly Media.
13. University of Ruhuna. (2026). ICT2132 Mini Project Guidelines.
14. W3Schools. (2025). Java Tutorial. Retrieved from https://www.w3schools.com/java/

## 10. Contribution

| Group member | Contribution |
|---|---|
| TG/2023/1753<br>Admin Module Developer & Integration Lead | Created the admin part and the UI for admin dashboard.<br>Implemented all admin functionalities including user profile management, course management, notice management, and timetable management.<br>Also checked the overall flow of the application and ensured seamless integration between modules. |
| TG/2023/1709<br>Lecturer Module Developer & System Designer | Created the lecturer part and its UI.<br>Implemented all lecturer functions including profile updates, course material uploads, marks entry, student details viewing, eligibility checking, marks/grades/GPA viewing, and attendance/medical records viewing.<br>Also created the UML class diagram and ER diagram for the database design. |
| TG/2023/1759<br>Student & Technical Officer Module Developer & Documentation Lead | Created the Student (Undergraduate) and Technical Officer (TO) parts along with their respective UIs.<br>Implemented all functions for both roles including attendance viewing, medical details viewing, course details viewing, grades/GPA viewing, timetables, notices (for students), and attendance/medical management (for TO).<br>Also created the complete project report and Software Requirement Specification (SRS) document. |